ENTERPRISE SYSTEM ARCHITECTURE // REPO: AMAZON-BEDROCK

AgentOS Studio & Bedrock Canvas

Comprehensive Platform Architecture, Full-Stack Implementation, ReAct Runtime Engine, Universal AI Graph Visualizer, and AWS Bedrock Converse Specifications.

> **Executive Overview: Amazon-BedRock (AgentOS)** is a state-of-the-art, enterprise-grade, full-stack autonomous multi-agent operating system. It provides an intuitive No-Code visual studio canvas for designing, versioning, testing, and deploying autonomous AI agent swarms backed by **Amazon Bedrock Foundation Models** (Anthropic Claude 3.5 Sonnet, Claude 3 Haiku, Amazon Nova Pro, Meta Llama 3) and connected to external **Model Context Protocol (MCP)** servers, REST APIs, and enterprise Knowledge Bases.

REPOSITORY LOCATION
D:\Projects\Amazon-BedRock

TECHNOLOGY STACK
FastAPI, React 18, Vite, TailwindCSS, Boto3, AWS CDK

BACKEND ARCHITECTURE
Control Plane Registry + ReAct Runtime Loop + MCP Router

FRONTEND ARCHITECTURE
Canvas Studio + Real-Time WebSocket Pub/Sub + Universal Graphs

AI FOUNDATION MODELS
AWS Bedrock Converse API + Zero-Cost Simulation Adapter

INFRASTRUCTURE AS CODE
AWS CDK (ECS Fargate, S3, IAM, CloudFormation)

Chapter 1: Platform Purpose & High-Level Architecture

1.1 What This Platform Actually Does

Building autonomous AI agents for enterprise environments is notoriously difficult. Developers face fragmented model APIs, complex authentication, lack of stateful memory, zero tool-safety guardrails, and no intuitive visual interface for business users to inspect agent reasoning steps.

AgentOS (Bedrock Canvas) solves these challenges by providing a cohesive, 3-tier operating system:

- **Tier 1: Studio Web Frontend (React/Vite on Port 3000):** A sleek, modern dashboard for designing agents, chatting in real-time with thought-streaming execution traces, managing sessions, and visualizing complex data via 9 interactive graph engines.
- **Tier 2: Control Plane & Governance (FastAPI on Port 8000):** Manages agent definitions, version tags (e.g. `1.0.0`), environment variables, groups, RAG knowledge bases, and performs AST static code analysis on MCP servers.
- **Tier 3: Autonomous Runtime Engine:** A recursive **ReAct (Reasoning + Acting)** execution loop that invokes Bedrock Converse models, resolves tool calls, isolates secret credentials, streams WebSocket events, and enforces budget token limits.

1.2 High-Level Architectural Dataflow

```
+-----------------------------------------------------------------------------------------------------+ | TIER 1: STUDIO WEB FRONTEND (apps/web) | | [Home/Explore Catalog] [Workspace Canvas] [Playground IDE] [Universal Graph Suite (9 Engines)] +-----------------------------------------------------------------------------------------------------+ | (HTTP REST API) | (WebSocket / SSE Pub/Sub) v v +-----------------------------------------------------------------------------------------------------+ | TIER 2: CONTROL PLANE & GOVERNANCE (apps/api) | | * FastAPI Application (main.py) * Central Registry (registry.py) | | * Natural Language Builder (nl_builder) * Static MCP Code Inspector (mcp_code_inspector) | | * Agent Versioning & Publishing * Multi-Session Thread Manager | +-----------------------------------------------------------------------------------------------------+ | | v v +-----------------------------------------------------------------------------------------------------+ | TIER 3: AUTONOMOUS RUNTIME ENGINE (services/runtime) | | +-----------------------------------------------------------------------------------------------------+ | | | Agent Runtime Engine (agent_loop.py) — Multi-Step ReAct Reasoning Loop | | | +-----------------------------------------------------------------------------------------------------+ | | | | | | | v v v | | [Model Router] [MCP Router] [Knowledge Router] | | * AWS Bedrock Converse API * stdio / SSE Transports * AWS Bedrock Knowledge Bases | | * Claude 3.5 Sonnet / Nova * {{MCP_AUTH_TOKEN}} Isolation * Local Vector Chunk Retrieval | | * Zero-Cost Local Adapter * Data Sanitization Engine * RAG Ingestion Pipeline | +-----------------------------------------------------------------------------------------------------+ | | | v v v [Amazon Bedrock Models] [MCP: mcp-market-intel] [S3 & Vector Databases]
```

Chapter 2: Codebase Structure & File Walkthrough

Here is the exact structure of `D:\Projects\Amazon-BedRock` and the role of every module:

```
d:\Projects\Amazon-BedRock/ ├── run.py # Master launcher: starts FastAPI (8000) & Vite (3000) concurrently ├── requirements.txt # Python dependencies (fastapi, uvicorn, boto3, cdk, httpx) ├── pyproject.toml # Build system, Ruff, and PyLint configuration ├── cdk.json # AWS Cloud Development Kit (CDK) project manifest ├── apps/ │ ├── api/ │ │ └── main.py # FastAPI Control Plane: REST routes, WebSockets, Pub/Sub bus │ └── web/ # React 18 / Vite / TailwindCSS Studio Frontend │ ├── src/App.jsx # Root application router and layout shell │ ├── src/components/ │ │ ├── HomeView.jsx # Platform dashboard, active agents, metric cards │ │ ├── ExploreView.jsx # Catalog of agents, tools, MCPs, and knowledge bases │ │ ├── WorkspaceView.jsx # Canvas for configuring agent identity, prompts, and tools │ │ ├── PlaygroundView.jsx # Live execution environment with thought streaming │ │ ├── ThreadsView.jsx # Multi-session conversation management & audit history │ │ ├── McpDetailView.jsx # Deep inspector for MCP servers and discovered capabilities │ │ ├── ToolDetailView.jsx # Tool configuration and parameter schema inspector │ │ └── graphs/ # Universal AI Data Visualization Suite │ │ ├── UniversalGraph.jsx # Graph orchestrator with auto-recommendation │ │ ├── aiGraphAdvisor.js # Dataset shape analysis & heuristic detection engine │ │ ├── CandlestickGraph.jsx # High-performance financial OHLCV candlestick engine │ │ ├── BarGraph.jsx # Multi-series comparative bar chart │ │ ├── LineAreaGraph.jsx # Time-series trend and area visualization │ │ ├── PieDonutGraph.jsx # Composition and portfolio allocation charts │ │ ├── ScatterBubbleGraph.jsx # Correlation and multi-dimensional bubble visualizer │ │ ├── RadarGraph.jsx # Multi-axis competency and risk radar │ │ ├── HeatmapGraph.jsx # Matrix correlation and density heatmap │ │ └── TableMatrixGraph.jsx # Structured tabular matrix data explorer ├── services/ │ ├── control_plane/ │ │ ├── registry.py # Central persistent store for agents, tools, models, & groups │ │ ├── nl_builder.py # Natural language prompt-to-agent synthesizer │ │ └── mcp_code_inspector.py # AST static code inspector generating Bedrock toolSpecs │ └── runtime/ │ ├── agent_loop.py # ReAct execution loop, recursive steps, token limiter │ ├── model_router.py # AWS Bedrock Converse API adapter + zero-cost simulation │ ├── mcp_router.py # MCP client router with {{MCP_AUTH_TOKEN}} masking │ ├── knowledge_router.py # Vector search and RAG knowledge base router │ ├── delegator.py # Multi-agent supervisor-worker delegation router │ ├── context_mgr.py # Thread memory, context sliding window, and pruning │ └── tool_runner.py # Python dynamic tool execution environment ├── packages/agent_spec/ │ └── models.py # Enterprise Pydantic v2 schemas (AgentDefinition, ToolSpec) └── infra/ ├── app.py # AWS CDK Application root └── ephemeral_stack.py # Ephemeral ECS Fargate & S3 stack for cloud sandboxes
```

Chapter 3: The 11 Enterprise Features You Have Built

1. Studio Web Frontend & Interactive Canvas (apps/web)

`FRONTEND / REACT 18`

What was built: A complete enterprise Web IDE engineered with React 18, Vite, and TailwindCSS. It provides a visual interface for operations teams without requiring code changes:

- **Explore Catalog:** Discover pre-configured enterprise agents (Financial Analyst, DevOps Engineer, Legal Auditor), registered MCP collections, and Bedrock models.
- **Workspace Canvas:** Configure agent personas, modular system prompts, attached tool whitelists, RAG knowledge bases, and scoped environment variables.
- **Threads & Sessions:** Preserves conversational history across sessions with full trace audit logs and latency/token counters.

2. The Universal AI Graph Visualization Suite (9 Chart Engines)

`DATA VISUALIZATION / SVG + CANVAS`

What was built: AI responses often return complex tabular or financial data that is unreadable as raw text. You built an entire visualization suite powered by `aiGraphAdvisor.js`:

GRAPH TYPE	ENGINE COMPONENT	AUTOMATIC DETECTION HEURISTIC
Candlestick	`CandlestickGraph.jsx`	Detects OHLCV keys (`open` , `high` , `low` , `close` , `volume`). Renders interactive financial candlestick charts with bullish/bearish color coding, volume bars, and crosshairs.
Bar & Column	`BarGraph.jsx`	Detects discrete categorical keys paired with numeric metrics.
Line & Area	`LineAreaGraph.jsx`	Detects date/time keys paired with continuous sequential metrics.
Pie & Donut	`PieDonutGraph.jsx`	Detects proportional compositions and portfolio sector allocations.
Scatter & Bubble	`ScatterBubbleGraph.jsx`	Detects dual numeric coordinate pairs (X, Y) with optional size scaling.
Radar / Spider	`RadarGraph.jsx`	Detects multi-attribute scoring across risk, performance, and capability.
Heatmap Matrix	`HeatmapGraph.jsx`	Detects two-dimensional correlation grids or density tables.
Table Matrix	`TableMatrixGraph.jsx`	Fallback for high-density multi-column data with sorting and search.

3. Natural Language Agent Builder (services/control_plane/nl_builder.py)

`AI COMPILER / GENERATIVE CONTROL PLANE`

What was built: Enables users to type a single conversational sentence (e.g. *"Create an equity researcher that inspects live prices and calculates portfolio risk"*) and automatically compiles an immutable `AgentDefinition`:

- Synthesizes role identity, professional guidelines, and structured system instructions.
- Matches and attaches relevant tools from the registry (e.g. web search, calculators, HTTP clients).
- Binds MCP collections (e.g. `mcp-market-intel`) and Bedrock foundation models automatically.

4. Static MCP Code Inspector (services/control_plane/mcp_code_inspector.py)

`STATIC ANALYSIS / AST COMPILER`

What was built: Connects to external Python or TypeScript MCP codebases (like `d:\Projects\MCPs\mcp-market-intel`) and inspects them safely without executing untrusted code:

- **Python AST Parsing:** Uses Python's native `ast` module to walk the code tree, discovering functions decorated with `@server.tool()` or `@mcp.tool()`.
- **Schema Synthesis:** Maps Python type annotations (`str`, `int`, `float`, `list`, `dict`) into standardized JSON Schema types.
- **Bedrock ToolSpec Generation:** Converts discovered MCP tools directly into authentic **AWS Bedrock Converse Tool Specifications** (`bedrockToolSpec.toolSpec`).
- **Auto-Discovery:** Scans sibling directories (`MCPs/`) and provides single-click connection to local MCP servers.

5. ReAct Autonomous Runtime Engine (services/runtime/agent_loop.py)

`REASONING CORE / REACT LOOP`

What was built: The heart of AgentOS. An execution loop that implements the **ReAct (Reasoning + Acting)** paradigm:

- **Recursive Execution:** Prompts the foundation model, inspects whether the model emitted text or a `toolUse` request, executes the target tool, feeds the `toolResult` back into the conversation, and repeats until the task is complete.
- **Budget Token Guardrails:** Tracks token consumption per execution step. If an agent exceeds `budgetLimitTokens`, the loop terminates safely, preventing runaway cloud costs.
- **HTML-to-Markdown Sanitizer:** Includes a custom regex converter (`html_to_markdown`) ensuring LLM outputs are clean GitHub-flavored Markdown without raw HTML tags.

6. Multi-Model Bedrock Converse Adapter (services/runtime/model_router.py)

`MODEL ABSTRACTION / AWS BEDROCK`

What was built: A unified adapter supporting Amazon Bedrock's modern Converse API:

- **Cross-Region Inference Profiles:** Automatically maps model identifiers to active AWS endpoints (e.g. `anthropic.claude-3-5-sonnet` ➔ `us.anthropic.claude-3-5-sonnet-20241022-v2:0`, `amazon.nova-pro` ➔ `us.amazon.nova-pro-v1:0`).
- **Zero-Cost Local Simulation:** If AWS credentials are not configured, the router switches seamlessly to a deterministic local simulation adapter, allowing developers to test multi-agent workflows with zero AWS charges.

7. MCP Secure Router & Credential Isolation (services/runtime/mcp_router.py)

`SECURITY / CREDENTIAL ISOLATION`

What was built: Dispatches tool executions to MCP servers while strictly protecting credentials:

- **Dynamic Token Interpolation:** Resolves templates like `Bearer {{MCP_AUTH_TOKEN}}` at execution time from Group variables or environment.
- **Bedrock Data Sanitization:** Strips sensitive tokens (`ghp_*`, `mcp_pat_*`, `sk_live_*`) and passwords from outputs before returning payloads to Bedrock or the user interface.

8–11. RAG Knowledge, Multi-Agent Swarms, WebSockets & AWS CDK `INFRASTRUCTURE / ORCHESTRATION`

- **8. Knowledge Router (`knowledge_router.py`):** RAG vector search routing to AWS Bedrock Knowledge Bases (`bedrock:Retrieve`) with chunk pre-fetching.
- **9. Multi-Agent Delegator (`delegator.py`):** Enables supervisor agents to delegate subtasks to specialized child agents with isolated execution traces.
- **10. Real-Time Pub/Sub Event Bus (`AgentStreamBroadcaster`):** WebSocket & SSE event broadcaster streaming live thoughts, tool executions, and status updates across client sessions.
- **11. AWS CDK Infrastructure (`infra/ephemeral_stack.py`):** CloudFormation stack provisioning S3 storage buckets, IAM execution roles, and Bedrock policies for automated preview sandboxes.

Chapter 4: Synergy with the MCP Market Intel Server

Your two repositories— `Amazon-BedRock` and `mcp-market-intel` —are designed to work as a seamless enterprise ecosystem:

```
+-------------------------------------------------------------------------------------------------+ | AGENTOS STUDIO (Amazon-BedRock) | | | | 1. MCP Code Inspector discovers D:\Projects\MCPs\mcp-market-intel | | 2. AST analyzes server.py and generates Bedrock toolSpec for: | | * yfinance_search * yfinance_get_quote * yfinance_get_stock_info | | * yfinance_get_historical * yfinance_get_news * portfolio_live_analysis | | 3. User creates "Wall Street Intelligence Agent" in WorkspaceView | +-------------------------------------------------------------------------------------------------+ | | (HTTP / SSE with Bearer {{MCP_AUTH_TOKEN}} on Port 8000/8001) v +-------------------------------------------------------------------------------------------------+ | MCP SERVER (mcp-market-intel) | | | | * Receives tool call: portfolio_live_analysis | | * Pulls live market prices, executes risk math, returns JSON metrics | +-------------------------------------------------------------------------------------------------+ | | (Returns JSON payload with OHLCV candles & P&L) v +-------------------------------------------------------------------------------------------------+ | PLAYGROUND & UNIVERSAL GRAPH VISUALIZER | | | | * aiGraphAdvisor detects OHLCV schema in toolResult | | * Instantly mounts CandlestickGraph.jsx & PieDonutGraph.jsx in the chat stream | | * User interacts with live zoomable candlestick charts and sector allocation donuts | +-------------------------------------------------------------------------------------------------+
```

The End-to-End Workflow in Action:

1. **Discovery:** You open AgentOS Studio at `http://localhost:3000`. In the MCP Collections tab, you click **Connect Local MCPs**. `mcp_code_inspector.py` inspects `d:\Projects\MCPs\mcp-market-intel`, parses all docstrings and type annotations, and creates the collection.
2. **Agent Creation:** You use the **Natural Language Builder**: *"Build an equity risk auditor that analyzes my portfolio"*. The builder attaches the `mcp-market-intel` tools and selects `anthropic.claude-3-5-sonnet`.
3. **Execution & Reasoning:** In the **Playground**, you ask: *"Here are my holdings in Apple and Tesla. Analyze my risk."* The ReAct engine invokes `portfolio_live_analysis`, resolving Bearer authentication transparently.
4. **Visual WOW Factor:** When the JSON returns, `UniversalGraph.jsx` intercepts the payload. Instead of dumping raw JSON text, the studio renders an interactive **Candlestick Chart** and a **Sector Allocation Donut** right in the chat conversation!

Chapter 5: Local Execution, Verification & Operations

5.1 Launching the Full Stack with a Single Command

The repository includes a unified launcher script (`run.py`) that boots both backend and frontend concurrently:

```
cd D:\Projects\Amazon-BedRock
python run.py
```

This initiates:

- **Control Plane & Runtime API:** `http://localhost:8000` (FastAPI with live reload).
- **Studio Web Frontend:** `http://localhost:3000` (Vite development server).
- **Interactive API Documentation:** `http://localhost:8000/docs` (Swagger UI).

5.2 Environment Configuration (.env)

Create or inspect `.env` in the repository root to configure cloud or local simulation modes:

```
# AWS Bedrock Credentials (Optional - switches to zero-cost local simulation if omitted)
AWS_ACCESS_KEY_ID=your_aws_access_key
AWS_SECRET_ACCESS_KEY=your_aws_secret_key
AWS_REGION=us-east-1

# Local MCP Projects Directory
MCP_PROJECTS_DIR=D:\Projects\MCPs
MCP_AUTH_TOKEN=sandbox-secret-key-12345
```

5.3 Quality Assurance & Code Standards

The repository adheres to strict enterprise linting and formatting standards configured in `pyproject.toml` :

TOOL	COMMAND	PURPOSE
Ruff Linter & Formatter	`ruff check . --fix`	Validates Python syntax, imports, and security practices in milliseconds.
PyLint	`python -m pylint services packages apps`	Enforces enterprise naming conventions, docstrings, and complexity limits.
Frontend ESLint	`cd apps/web && npm run lint`	Validates React JSX component syntax and hook dependency arrays.
SonarQube Analysis	`docker-compose -f docker-compose.sonar.yml up -d`	Executes continuous code quality, security vulnerability, and coverage audits.

> **Architecture Summary: Amazon-BedRock (AgentOS)** is an enterprise multi-agent operating system. It features a complete visual React frontend, an AST-based MCP code inspector, a ReAct reasoning engine, multi-model Bedrock routing, token security isolation, 9 universal interactive graph engines, and cloud infrastructure as code.